



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION



School of Engineering & Technology

Lab Assignment -1 Algorithm Foundations Instructions

Course Name:

Analysis and Design of Algorithms Lab

Course Code:

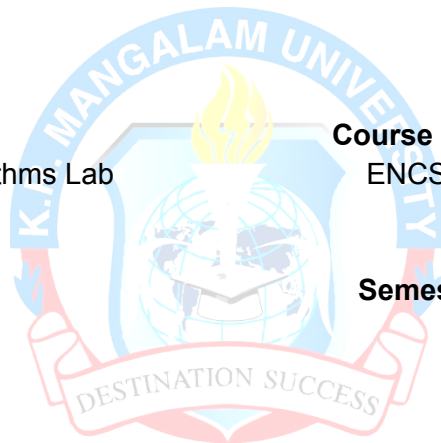
ENCS256

Name: BADAL PRASAD

Roll No. :- 2501351020

Semester: IV

Github Link : [Assignment 01](#)



Assignment Title: Understanding Algorithm Efficiency through Analysis, Recurrences, and Sorting

Real-World Problem Context

In real-world software systems, efficiency is as important as correctness. Whether designing a search engine, database engine, or embedded system, developers must evaluate algorithms based on their **time and space** behaviour rather than intuition alone. This assignment introduces foundational concepts such as algorithm analysis, asymptotic notations, **recurrence relations, and sorting techniques**, enabling students to reason about scalability and performance before implementation.

Q1. Task 1: Algorithm Growth Observation Write Python programs to implement the following functions:

1. Constant Time : $O(1)$
2. Linear Time : $O(n)$
3. Quadratic Time : $O(n^2)$
4. Logarithmic Time : $O(\log n)$

CODE :

```
/*Task 1: Algorithm Growth Observation
Write Java programs to implement the
following functions: */

package assignment01;

public class TimeComplexity {

    // ① Constant Time  $O(1)$ 
    public static int getFirst(int[] arr) {
        return arr[0];
    }

    // ② Linear Time  $O(n)$ 
    public static int sumArray(int[] arr) {
        int sum = 0;
        for (int i = 0; i < arr.length; i++) {
            sum += arr[i];
        }
    }
}
```

```

    }
    return sum;
}

// ③ Quadratic Time  $O(n^2)$ 
public static void printPairs(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        for (int j = 0; j < arr.length; j++) {
            int x = arr[i] + arr[j]; // dummy
work
        }
    }
}

// ④ Logarithmic Time  $O(\log n)$ 
public static int binarySearch(int[] arr, int
key) {
    int low = 0, high = arr.length - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
}

```

```
    }  
    return -1;  
}  
  
public static void main(String[] args) {  
  
    int[] sizes = {10, 100, 500, 1000};  
  
    for (int n : sizes) {  
  
        int[] arr = new int[n];  
        for (int i = 0; i < n; i++)  
            arr[i] = i;  
  
        // O(1)  
        long start1 = System.nanoTime();  
        getFirst(arr);  
        long end1 = System.nanoTime();  
  
        // O(n)  
        long start2 = System.nanoTime();  
        sumArray(arr);  
        long end2 = System.nanoTime();  
  
        // O(n^2)  
        long start3 = System.nanoTime();  
        printPairs(arr);  
        long end3 = System.nanoTime();  
    }  
}
```

```

        // O(log n)
        long start4 = System.nanoTime();
        binarySearch(arr, n - 1);
        long end4 = System.nanoTime();

        System.out.println("n = " + n);
        System.out.println("O(1) Time: " +
(end1 - start1));
        System.out.println("O(n) Time: " +
(end2 - start2));
        System.out.println("O(n^2) Time: " +
(end3 - start3));
        System.out.println("O(log n) Time: " +
(end4 - start4));

System.out.println("-----");
    }
}

/*
n = 10
O(1) Time: 1200
O(n) Time: 1100
O(n^2) Time: 2400
O(log n) Time: 1200
-----
n = 100

```

```
O(1) Time: 400
O(n) Time: 1200
O(n^2) Time: 117000
O(log n) Time: 700
-----
n = 500
O(1) Time: 300
O(n) Time: 4900
O(n^2) Time: 1159400
O(log n) Time: 1300
-----
n = 1000
O(1) Time: 500
O(n) Time: 12100
O(n^2) Time: 1697000
O(log n) Time: 900
----- */
```

Q2.Task 2: Best, Average, and Worst Case Analysis

Implement **Linear Search** and **Binary Search**.

Requirements

- Generate random arrays of different sizes.
- Measure execution time for:
 - Best case
 - Average case
 - Worst case
- Compare results using plots.

Deliverables

- Search algorithm implementations
- Comparison plots
- Observations in comments/markdown

CODE: /*Task 2: Best, Average, and Worst Case Analysis

```
Implement Linear Search and Binary Search*/

package assignment01;

import java.util.Arrays;
import java.util.Random;

public class SearchAnalysis {

    // ----- Linear Search -----
    public static int linearSearch(int[] arr, int
key) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key)
                return i;
        }
    }
}
```

```
        return -1;
    }

    // ----- Binary Search -----
    public static int binarySearch(int[] arr, int
key) {
        int low = 0, high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (arr[mid] == key)
                return mid;
            else if (arr[mid] < key)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return -1;
    }

    // ----- Generate Random Array -----
    public static int[] generateArray(int n) {
        Random rand = new Random();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = rand.nextInt(n * 10);
        }
    }
}
```



```

    }
    return arr;
}

// ----- Time Measurement -----
public static long measureLinear(int[] arr, int
key) {
    long start = System.nanoTime();
    linearSearch(arr, key);
    long end = System.nanoTime();
    return end - start;
}

public static long measureBinary(int[] arr, int
key) {
    long start = System.nanoTime();
    binarySearch(arr, key);
    long end = System.nanoTime();
    return end - start;
}

// ----- Main -----
public static void main(String[] args) {

    int[] sizes = {10, 100, 500, 1000};

    // Table Header

```

```

        System.out.printf("%-6s %-12s %-12s %-12s
%-12s %-12s %-12s%n",
            "Size", "LinearBest", "LinearAvg",
"LinearWorst",
            "BinaryBest", "BinaryAvg",
"BinaryWorst");

        for (int n : sizes) {

            int[] arr = generateArray(n);
            Arrays.sort(arr); // required for
binary search

            int bestKey = arr[0];           // first
element
            int avgKey = arr[n / 2];        // middle
element
            int worstKey = -1;              // not
present

            long lBest = measureLinear(arr,
bestKey);
            long lAvg = measureLinear(arr, avgKey);
            long lWorst = measureLinear(arr,
worstKey);

            long bBest = measureBinary(arr,
bestKey);

```

```

        long bAvg = measureBinary(arr, avgKey);
        long bWorst = measureBinary(arr,
worstKey);

        System.out.printf("%-6d %-12d %-12d
%-12d %-12d %-12d %-12d%n",
            n, lBest, lAvg, lWorst, bBest,
bAvg, bWorst);
    }
}
}

```

```

/* Size    LinearBest    LinearAvg    LinearWorst
BinaryBest    BinaryAvg    BinaryWorst
10        700        200        200        800
300
200
100        400        600        1000        500
200        200
500        600        4000        7700        800
500        400
1000       400        8100        14900        800
100        400

2501351020
Badal*/

```

Q3.Task 3: Recursion and Recurrence Validation Implement the following recursive algorithms:

1. Factorial
2. Fibonacci (Naïve Recursive)
3. Fibonacci (Dynamic Programming)

Steps

- Count number of function calls for each input size.
- Measure execution time.
- Compare recursive vs optimized approach.

Deliverables

- Python/ JAVA implementations
- Table comparing execution time and function calls

CODE:// Task 3: Recursion and Recurrence Validation

```
package assignment01;

import java.util.HashMap;

public class RecursionValidation {

    // ----- Call Counters -----
    static int factCalls = 0;
    static int fibCalls = 0;
    static int fibDpCalls = 0;

    // ----- Factorial -----
    public static long factorial(int n) {
        factCalls++;

        if (n <= 1)
            return 1;
    }
}
```

```

        return n * factorial(n - 1);
    }

    // ----- Fibonacci (Naïve) -----
    public static long fibonacci(int n) {
        fibCalls++;

        if (n <= 1)
            return n;

        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    // ----- Fibonacci (DP Memoization)
    -----
    public static long fibonacciDP(int n,
    HashMap<Integer, Long> memo) {
        fibDpCalls++;

        if (memo.containsKey(n))
            return memo.get(n);

        if (n <= 1)
            return (long) n;

        long value = fibonacciDP(n - 1, memo) +
    fibonacciDP(n - 2, memo);
        memo.put(n, value);
    }

```

```
        return value;
    }

    public static void main(String[] args) {

        int[] sizes = {5, 10, 20, 30};

        System.out.printf("%-6s %-12s %-12s %-12s
%-12s %-12s %-12s%n",
            "n", "FactCalls", "FactTime",
            "FibCalls", "FibTime",
            "FibDPCalls", "FibDPTime");

        for (int n : sizes) {

            factCalls = fibCalls = fibDpCalls = 0;

            // Factorial
            long start = System.nanoTime();
            factorial(n);
            long factTime = System.nanoTime() -
start;

            // Fibonacci naive
            start = System.nanoTime();
            fibonacci(n);
            long fibTime = System.nanoTime() -
start;
```

```

        // Fibonacci DP
        start = System.nanoTime();
        fibonacciDP(n, new HashMap<>());
        long fibDpTime = System.nanoTime() -
start;

        System.out.printf("%-6d %-12d %-12d
%-12d %-12d %-12d %-12d%n",
                        n, factCalls, factTime,
fibCalls, fibTime, fibDpCalls, fibDpTime);
    }
}

```

```

/*
n      FactCalls      FactTime      FibCalls
FibTime      FibDPCalls      FibDPTime
5      5      1200      15      1000
9      61800
10     10      1200      177      5200
19     29400
20     20      1200      21891
295900      39      31700
30     30      10300      2692537
3445500      59      53100 */

```

Q4: Task 4: Solving Recurrences through Code Implement recursive functions corresponding to these recurrences:

1. $T(n) = T(n/2) + n$
2. $T(n) = 2T(n/2) + n$

Steps

- Count recursive calls.
- Measure execution time.
- Validate results with expected asymptotic complexity.

Deliverables

- Code with call counters
- Observational comments

CODE: //Task 4: Solving Recurrences through Code

```
package assignment01;

public class RecurrenceAnalysis {

    // Call counters
    static int callsT1 = 0;
    static int callsT2 = 0;

    // ----- Recurrence 1 -----
    //  $T(n) = T(n/2) + n \rightarrow O(n)$ 
    public static int T1(int n) {
        callsT1++;

        if (n <= 1)
            return 1;

        // linear work
        int work = 0;
```



```

        for (int i = 0; i < n; i++)
            work++;

        return T1(n / 2) + work;
    }

    // ----- Recurrence 2 -----
    //  $T(n) = 2T(n/2) + n \rightarrow O(n \log n)$ 
    public static int T2(int n) {
        callsT2++;

        if (n <= 1)
            return 1;

        int work = 0;
        for (int i = 0; i < n; i++)
            work++;

        return T2(n / 2) + T2(n / 2) + work;
    }

    public static void main(String[] args) {

        int[] sizes = {8, 16, 32, 64, 128, 256};

        System.out.printf("%-6s %-12s %-14s %-12s\n",
%-14s%n",

```

```
        "n", "CallsT1", "TimeT1(ns)",
"CallsT2", "TimeT2(ns)");

    for (int n : sizes) {

        callsT1 = 0;
        callsT2 = 0;

        long start1 = System.nanoTime();
        T1(n);
        long time1 = System.nanoTime() -
start1;

        long start2 = System.nanoTime();
        T2(n);
        long time2 = System.nanoTime() -
start2;

        System.out.printf("%-6d %-12d %-14d
%-12d %-14d%n",
                        n, callsT1, time1, callsT2,
time2);
    }
}
```

```

/*n      CallsT1      TimeT1 (ns)      CallsT2
TimeT2 (ns)
8        4            1500            15
1100
16       5            1100            31
1700
32       6            1100            63
3100
64       7            2200            127
10300
128      8            3500            255
28000
256     9            4000            511
13300
*/

```





